

picoCTF

Challenge Writeups

Reverse Engineering · Web Exploitation · Steganography

Rob — 2026

All flags are partially redacted. Characters inside [...] have been obscured to preserve the challenge for future solvers.

Challenge 1: m00nwalk2

Category	Steganography / Forensics
Tools	Audacity, sstv-decoder, steghide
Difficulty	Medium

Overview

A set of WAV audio files was provided alongside hint images. The goal was to extract hidden data from the carrier audio. This challenge stacked two steganography techniques: SSTV image encoding in audio and steghide data hiding in WAV files.

Solution

Step 1 — SSTV Decode

The WAV files were decoded as Slow-Scan Television (SSTV) transmissions using a software-defined radio decoder. This revealed a set of images, one of which contained a clearly visible steganographic password hint.

```
python3 sstv_decode.py clue1.wav
```

The decoded image showed a stegosaurus silhouette alongside the text 'hidden_stegosaurus'.

Step 2 — Steghide Extraction

With the passphrase identified from the SSTV image, steghide was used to extract the hidden payload from the carrier WAV file:

```
steghide extract -sf message.wav -p "hidden_stegosaurus"
```

This produced a text file containing the flag.

Key Takeaway

Multi-layer steganography is common in CTF challenges. Always decode all provided media files and look for hidden metadata, embedded images, or strings that could serve as passwords for the next layer.

Flag: picoCTF{th3_[...]_l135_h1dd3n_[...]_pl41n_[...]}

Challenge 2: SQL Injection — Login Bypass

Category	Web Exploitation
Tools	Browser / curl
Difficulty	Easy

Overview

A web application presented a login form. The goal was to bypass authentication without valid credentials by injecting SQL into the username field.

Solution

Injection Payload

Classic SQL injection was applied to the username field. The payload terminates the intended query with a single quote, injects an admin username, and comments out the remainder of the SQL statement including the password check:

```
Username: admin'--  
Password: (anything)
```

The resulting SQL executed on the server became:

```
SELECT * FROM users WHERE username='admin'--' AND password='...'
```

The double-dash comment causes the password clause to be ignored entirely, granting access as the admin user.

Key Takeaway

Input fields that are concatenated directly into SQL queries without parameterisation are trivially exploitable. Always use prepared statements with bound parameters in production code.

Flag: picoCTF{[...redacted...]}

Challenge 3: SQL Injection — Hardened Bypass

Category	Web Exploitation
Tools	Browser / curl
Difficulty	Medium

Overview

A strengthened version of the login bypass challenge. The server appeared to apply some transformation to credentials before inserting them into the SQL query, making a direct injection fail.

Solution

Identifying the Transformation

Testing revealed the server was applying ROT13 to the submitted credentials before passing them to the SQL query. To craft a payload that reads as SQL injection after ROT13 decoding, the input itself had to be the ROT13 of the intended SQL string:

```
ROT13('nqzva\'--') => admin'--
```

The username field was therefore submitted as the ROT13-encoded version of the injection payload. After the server-side ROT13 transformation, the query received the classic bypass string.

Key Takeaway

Obfuscation of inputs (ROT13, base64, etc.) is not a security control. If the transformation is known or discoverable, an attacker simply pre-transforms their payload. Parameterised queries are the only reliable defence.

Flag: picoCTF{[...redacted...]}

Challenge 4: SQL Injection — Admin Login

Category	Web Exploitation
Tools	Browser / curl
Difficulty	Easy–Medium

Overview

Another authentication bypass challenge requiring a login as the admin user. The application had a different schema from the prior challenges.

Solution

The same family of SQL injection payload was effective here. The single-quote comment bypass was applied directly to the username field:

```
Username: admin'--
```

Upon successful bypass, the application returned the flag in the authenticated session response.

Key Takeaway

Variations of the same vulnerability appear across challenges at different difficulty ratings. Methodical testing of injection points — even on applications that appear hardened — is always warranted.

Flag: picoCTF{[...redacted...]}

Challenge 5: mystery (Binary RE)

Category	Reverse Engineering
Tools	radare2, Python 3
Difficulty	Hard
Note	Flag is NOT in picoCTF{} format

Overview

A stripped ELF 64-bit binary and an encoded output file were provided. The binary reads flag.txt, encodes its content using a bespoke bit-packing scheme, and writes the result to a file named output. The task was to reverse the encoding and recover the original plaintext.

Reverse Engineering

Encoding Scheme

Disassembly with radare2 revealed three key data structures embedded in the binary's .rodata section:

- secret — a 37-byte bitstream containing unique bit patterns for each character
- matrix — 26 pairs of (count, start_bit) defining the bit slice within secret that encodes each letter a–z

The encode() function maps each character to its corresponding slice of the secret bitstream, then packs those bits MSB-first into the output file using a one-bit-at-a-time write loop. A global remain counter (initialised to 7) tracks the position within the current output byte.

Decoding

A Python decoder was written to:

- Extract all character bit patterns from the secret and matrix structures
- Convert the output file to a flat bit stream
- Greedy match the bit stream against known patterns (shortest match first — the code is prefix-free)
- Accumulate matched characters until the stream is exhausted

```
# Greedy decode (Python)
patterns = { bits_tuple: char }
pos = 0
while pos < len(out_bits):
    for length in range(1, max_len+1):
        if tuple(out_bits[pos:pos+length]) in patterns:
            result.append(patterns[candidate])
            pos += length; break
```

The decoded plaintext was re-encoded to verify it reproduced the provided output file byte-for-byte.

Flag: enc[...]yk[...]

Challenge 6: no_auth_mystery (Binary RE + Auth Bypass)

Category	Reverse Engineering
Tools	radare2, Python 3, GCC (LD_PRELOAD)
Difficulty	Hard
Note	Flag is NOT in picoCTF{} format — lowercase + digits only

Overview

A more complex variant of the mystery binary. This ELF adds a network authentication layer: before encoding, `login()` connects to `fakeauthsite.com` on port 2345, sends a credential string, receives a response, and `strcmp()`s it against a hardcoded token. A mismatch causes the binary to exit before encoding. An encoded output file was also provided.

Reverse Engineering

Authentication Layer

Disassembly of `login()` revealed:

- `gethostbyname('ZmFrZWZlZGhzaXRl.com')` — the hostname is the base64 encoding of 'fakeauthsite'
- `connect()` to port 0x929 (2345)
- `send()` of a 100-byte Base64 credential string
- `recv()` of 33 bytes compared via `strcmp()` against 'QXV0aG9yaXplZCB0byBleGVjdXRlLi4u'
- `je 0x1028` — conditional jump to the authorised path on equality

Patching the Auth Check

The conditional jump at file offset 0xFF8 was patched from `je (0x74)` to `jmp (0xEB)` so the binary always follows the authorised path regardless of the network response:

```
# Python one-liner patch data = bytearray(open('no_auth_mystery', 'rb').read())
data[0xFF8] = 0xEB # je -> jmp open('no_auth_mystery_patched', 'wb').write(data)
```

Encoding Scheme (Differences from mystery)

The encoding scheme is similar to `mystery` but with two notable differences:

- `secret` — 70 bytes (560 bits) stored at 0x1300, covering a-z, 0-9, and space
- `indexTable` — 38 int32 start offsets at 0x1360; each character's pattern runs from `indexTable[slot]` to `indexTable[slot+1]`
- Character rotation — the slot index for character `c_idx` is $(c_idx + 18) \% 36$, not a direct index

The rotation `slot = (c_idx + 18) % 36` is compiled as a fixed-point division-by-36 multiply with magic constant `0x38E38E39` followed by a right shift.

The `save()` function packs bits MSB-first with `remain` initialised to 7, writing a complete byte when `remain` reaches 0 after the 8th bit.

Decoding

The same greedy prefix-free approach used in mystery was applied, with the rotation formula inverted. The trailing four bits of the output were the encoding of a trailing space in flag.txt and were discarded. The decoded plaintext was verified by re-encoding to the exact output file bytes.

Flag: t1m3f1[...]000[...]17b

Summary

Challenge	Category	Difficulty	Technique
m00nwalk2	Steganography	Medium	SSTV decode → steghide extract
SQL Inject. 1	Web Exploitation	Easy	admin'-- comment injection
SQL Inject. 2	Web Exploitation	Medium	Pre-encode payload in ROT13
SQL Inject. 3	Web Exploitation	Easy	admin'-- comment injection
mystery	Reverse Engineering	Hard	Bit-pattern decode + greedy match
no_auth_mystery	Reverse Engineering	Hard	je → jmp patch + rotation decode